MATERIAL DE APRENDIZAJE · PYTHON

Funciones en Python

Definición, parámetros, llamadas y organización de programas

Ing. William Ramón Flórez · Instructor SENA
Visual Studio Code & Python

Una guía práctica construida a partir de los ejercicios de la carpeta **funciones**. Incluye los programas, su propósito y los conceptos que se aplican en cada uno.

Temario principal

- Qué es una función y cómo se escribe
- Funciones sin parámetros
- Funciones con parámetros y datos ingresados por teclado
- Ejercicios integradores: varias funciones y decisiones
- Buenas prácticas y repaso

Propósito: reconocer cómo una función agrupa instrucciones bajo un nombre y cómo se ejecuta al invocarla.

1. ¿Qué es una función?

Una función es un bloque de instrucciones con nombre que puede ejecutarse cuando el programa la llama. En Python se define con `def`, seguido del nombre, paréntesis y dos puntos. El bloque se escribe con sangría.

```
def nombre_de_funcion():
    instrucciones

nombre_de_funcion()
```

- `def` inicia la definición.
- El nombre describe la tarea que realiza.
- Los paréntesis pueden estar vacíos o contener parámetros.
- La llamada se escribe con el nombre y paréntesis: la definición por sí sola no ejecuta el bloque.

En estos ejercicios: las funciones muestran mensajes con `print()`. No usan `return`; por eso el resultado se presenta directamente en la consola.

2. Funciones sin parámetros

Estas funciones no reciben datos entre sus paréntesis. Cada llamada realiza la tarea definida, por ejemplo, mostrar un mensaje, una ficha o reglas de trabajo.

Mensaje de bienvenida

`bienvenida.py`

La función encapsula un mensaje fijo y se ejecuta al final del archivo.

```
def bienvenida_python():
    print("Iniciando motor de ejecución de Python 3.x")

bienvenida_python()
```

Programa de formación

`saludo.py`

La variable se crea dentro de la función; su alcance queda limitado a ese bloque.

```
def mostrar_programa():
    programa="ADSO"
    print(f"Bienvenido al programa: {programa}")

mostrar_programa()
```

Salida: Bienvenido al programa: ADSO

Mostrar número de ficha

`ficha.py`

```
def mostrar_ficha():
    ficha = 3388070
    print(f"Ficha: {ficha}")

mostrar_ficha()
```

Aviso de mantenimiento

`mantenimiento.py`

```
def aviso_mantenimiento():
    print("El programa se encuentra en mantenimiento, por favor intente más tarde.")

aviso_mantenimiento()
```

Reglas del ambiente

normas.py

```
def reglas_ambiente():
    print("1. Mantener el área de trabajo limpia y ordenada.")
    print("2. No consumir alimentos ni bebidas cerca del equipo.")

reglas_ambiente()
```

Reporte final

reporte.py

```
def reporte_final():
    print("Práctica de funciones concluida satisfactoriamente")

reporte_final()
```

Respaldo de datos

respaldo.py

El texto del ejercicio menciona archivos PHP y Python; la función únicamente presenta el aviso, no realiza una copia real.

```
def respaldo_datos():
    print("Generando copia de seguridad de archivos PHP y Python...")

respaldo_datos()
```

Estado de conexión

cloud.py

```
def estado_cloud():
    print("Conexión estable con el servidor remoto de VS Code")

estado_cloud()
```

3. Parámetros y entradas

Un parámetro es una variable incluida en la definición de una función para recibir un dato. Al llamar la función se entrega un argumento, que ocupa el lugar de ese parámetro.

```
def saludar(nombre):
    print(f"Hola, {nombre}")

saludar("Ana")
```

Presentación personal

`presentacion.py`

El programa solicita el nombre, define el rol y envía ambos valores como argumentos a la función.

```
nombre = input("Ingrese su nombre completo: ")
rol = "aprendiz ADSO"

def presentacion_personal(nombre, rol):
    print(f"Bienvenido {nombre}, eres {rol}")

presentacion_personal(nombre, rol)
```

Saludo por trimestre

`trimestre.py`

El valor ingresado por teclado se pasa a la función como argumento.

```
trimestre = input("Ingrese su trimestre actual: ")

def saludar_trimestre(trimestre):
    print(f"Bienvenido al trimestre: {trimestre}")

saludar_trimestre(trimestre)
```

4. Ejercicios integradores

Al dividir un programa en funciones, cada bloque puede encargarse de una parte concreta. Esto facilita leer el flujo, reutilizar instrucciones y mantener el código organizado.

Datos de la institución

`ejercicio_integrado1.py`

Se definen funciones para mostrar el encabezado, los detalles del programa y el pie de página. La respuesta del usuario decide cuáles se llaman.

```
n_insitucion = "SENA"
n_centro = "Centro de servicios financieros"
instructor = "William Ramón Flórez"
n_programa = "ADSO"

def mostrar_encabezado():
    print(f"Institución: {n_insitucion}")
    print(f"Centro de formación: {n_centro}")

def mostrar_detalles_programa():
    print(f"Instructor: {instructor}")
    print(f"Programa: {n_programa}")

def mostrar_pie_pagina():
    print("Gracias por utilizar este programa de funciones en Python")

mostrar_encabezado()
detalles = input("mostrar detalles (s/n): ")
if detalles.lower() == "s":
    mostrar_detalles_programa()
    mostrar_pie_pagina()
elif detalles.lower() == "n":
    mostrar_pie_pagina()
else:
    print("Opción no válida. Por favor, ingrese 's' o 'n'.")
```

Menú de misión, visión y contacto

`ejercicio_integrado2.py`

Este ejemplo combina datos compartidos, funciones pequeñas, una entrada de usuario y una estructura `if/elif/else`. El bloque principal evita que el menú se ejecute cuando el archivo se importa desde otro módulo.

```
mision = "cumplir la función que le corresponde al Estado de invertir en el desarrollo social y técnico de los
trabajadores colombianos."
vision = "ser una entidad de excelencia en la formación integral de los aprendices, con calidad y pertinencia, que
contribuya al desarrollo del país."
contacto = "https://www.sena.edu.co/es-co/Paginas/contactenos.aspx"

def opcion_mision():
    print(f"Misión: {mision}")

def opcion_vision():
    print(f"Visión: {vision}")

def opcion_contacto():
    print(f"Contacto: {contacto}")

if __name__ == "__main__":
    print("Opciones disponibles:")
    print("1. Misión")
    print("2. Visión")
    print("3. Contacto")
    opcion = input("Seleccione una opción (1, 2 o 3): ")

    if opcion == "1":
        opcion_mision()
    elif opcion == "2":
        opcion_vision()
    elif opcion == "3":
        opcion_contacto()
    else:
        print("Opción no válida.")
```

5. Ideas clave y buenas prácticas

- Usa nombres descriptivos, normalmente en minúsculas y separados por guion bajo: `mostrar_ficha()`.
- Define la función antes de la llamada, para que Python conozca su nombre cuando deba ejecutarla.
- Mantén la sangría consistente dentro del bloque de la función.
- Pasa como argumentos los datos que la función necesita; así puede trabajar con distintos valores.
- Divide programas largos en funciones que realicen tareas concretas.
- Usa `if __name__ == "__main__":` cuando el archivo también pueda importarse desde otro programa.

Repaso: ¿qué palabra inicia una función?, ¿cómo se llama una función sin parámetros?, ¿qué diferencia hay entre un parámetro y un argumento?